GPU-Accelerated Monte Carlo Ray Tracing with BVH Optimization

Kaveh Sanaei

M.Sc. Computer Science — TU Berlin

Objectives

- Implement a CUDA-based **Monte Carlo path tracer**.
- Accelerate intersections using a **Bounding Volume Hierarchy (BVH)**.
- Apply **GPU optimizations**.
- Evaluate runtime using **Nsight Compute profiling**.

Introduction

Ray tracing simulates physical light transport to generate realistic reflections, refractions, shadows, and global illumination. This project develops a **GPU-accelerated Monte Carlo path tracer in CUDA** using **data-oriented design**, **BVH spatial acceleration**, and **Russian roulette termination** to achieve physically accurate rendering with high computational efficiency.

Monte Carlo Integration

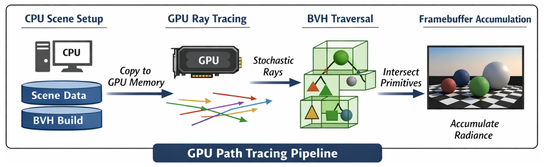
Global illumination is estimated via **stochastic Monte Carlo sampling**. Multiple random rays are traced per pixel, with probabilistic material scattering and averaging of radiance samples to produce an **unbiased solution** of the rendering equation. Russian roulette termination reduces average ray depth while preserving correctness and improving runtime.

GPU Rendering Architecture

- **Parallel execution:** one CUDA thread computes one pixel sample.
- **Random sampling:** CURAND provides stochastic sampling for Monte Carlo integration and material scattering.
- **Main CUDA kernels:**
  - `render_init`: initialize per-pixel random states.
  - `render`: perform Monte Carlo path tracing in parallel.
  - `ray_color`: iterative light transport with BVH traversal and Russian roulette termination.

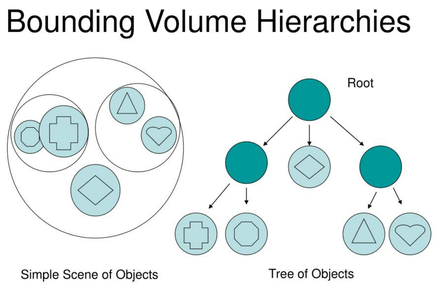
Runtime Pipeline

1. CPU constructs the scene and builds the BVH.
2. Scene and BVH data are copied to GPU memory.
3. CUDA initializes a random state per pixel.
4. Each thread traces stochastic rays through the BVH.
5. Rays bounce via probabilistic material scattering.
6. Radiance samples are averaged into a linear framebuffer.



BVH Acceleration

A **Bounding Volume Hierarchy** groups geometry into hierarchical bounding boxes so rays intersect boxes before primitives. This reduces intersection complexity from $\mathbf{O(N)}$ to $\approx O(\log N)$, providing the largest algorithmic speedup. The BVH is built on the CPU and traversed inside CUDA kernels.



Ray–Sphere Intersection

A ray is

$$r(t) = \mathbf{o} + t\mathbf{d}, \quad \|\mathbf{p} - \mathbf{c}\|^2 = R^2.$$

This yields

$$at^2 + bt + c = 0, \quad \Delta = b^2 - 4ac.$$

A hit occurs when $\Delta \geq 0$ and the root lies in the valid ray interval.

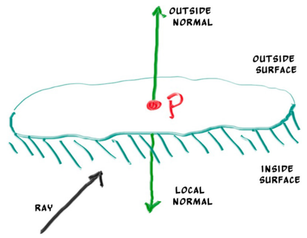


Figure 1:Ray–Sphere Intersection

Optimization Strategies

- **Data-oriented design:** runtime reduced from **40 s** to **21 s** by removing virtual dispatch and improving memory locality.
- **BVH baseline:** 6.7 s runtime, 84 registers, 37% occupancy, 39% compute throughput.
- **Explicit device memory: 6.7 s → 5.3 s** (23% faster).
- **Register reduction: 84 → 66 registers**, occupancy **52%**, runtime **4.4 s**.
- **Russian roulette: 4.4 s → 3.0 s** (32% faster).

Nsight profiling after Russian roulette: The main `render` kernel is **well-balanced** with about **76% compute throughput** and **76% memory throughput**, but remains **register-limited** (**70 registers per thread**), which caps theoretical occupancy at **58.3%** (achieved **53.8%**).

Final true runtime: 1.73 s (without profiling overhead).

Output

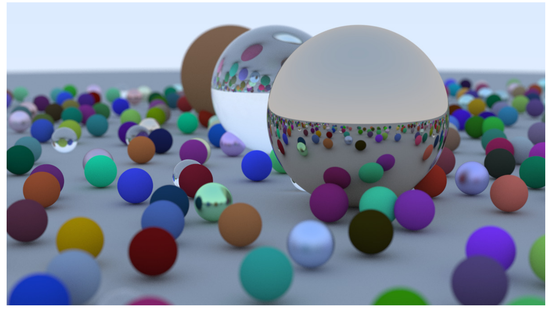


Figure 2:Final render.

Rendering setup

- Resolution: **1200 × 675** (16:9)
- Samples per pixel: **500**
- Max path depth: **20**
- Scene: **487 spheres / 487 materials**
- Block size: **8 × 8**

Performance

- True runtime: **1.73 s**
- Nsight profiled: ≈ **3.0 s** (profiling overhead)

Conclusion

A CUDA Monte Carlo ray tracer was implemented using **data-oriented design, BVH acceleration, register-pressure reduction, and Russian roulette termination**. Runtime decreased from **40 s to 1.73 s** and compute throughput from **39% to 76%** while preserving physically accurate global illumination, demonstrating the importance of **algorithmic acceleration and GPU-aware optimization**.

Contact

- Kaveh Sanaei
- M.Sc. Computer Science — TU Berlin
- Email: `kaveh.sanaei@campus.tu-berlin.de`